VERIQO CRE + VTECP-001 ? CLOSURE REPORT

Assessment date: 28 August 2026
Baseline: R11 (FINAL INTERNAL CHECK) deliverable ? commit 0540117
Responds to: three attached specification documents ? VERIQO_CASE_RESOLUTION_ENGINE.docx (CRE), VERIQO_TRUST_EVIDENCE_CONTROL_PLANE.docx (VTECP-001), and Bagian_dari_Case_Resolution_Engine.docx (a framing memo extending both) ? plus the explicit instruction to build the Case Resolution Engine "serta beberapa fondasi lainnya" (plus some other foundations), in as much detail as possible, with no remaining gap, efficiently, with thorough integration testing.
Assessment mode: local engineering qualification and reproducibility review. All code in this deliverable compiles, is tested, and was verified against the live repository ? no external system, counterparty, or credential was contacted or fabricated.

EXECUTIVE VERDICT

Both specifications describe a broad, five-capability trust and evidence platform (VTECP-001) and an eleven-stage case resolution pipeline (CRE) sitting on top of it. A repository survey early in this round found something the specifications themselves did not anticipate: almost every CRE pipeline stage, and three of VTECP-001's five capabilities, already existed in this ~700-file repository, built and tested across ten prior rounds of work on the insurance domain.
Building a second, parallel implementation of any of it would have been pure duplication ? exactly what both specs' own instructions forbid ("Claude should first inspect the existing repository and reuse existing packages wherever possible... If equivalent packages already exist, Claude MUST extend them rather than duplicate functionality").

So this round's real work was threefold:

1. Build the two capabilities that had no existing equivalent at all ? RFC 8785 JSON canonicalization (a stated prerequisite for everything else), the Immutable Evidence Manifest, the generic Ontology object/link/action registry, and a Merkle inclusion-proof + anchoring layer over the audit ledger.
2. Extend, not duplicate, the capabilities that were 80% there ? pkg/authz already had real RBAC+ABAC+ReBAC; it gained Purpose Binding and a canonical PolicyDecision record. pkg/governance/lifecycle already gated a model through DRAFT?...?ACTIVE; a new pkg/inference package uses it as the enforcement point for a generic InferenceTrace.
3. Build the one genuinely missing CRE artifact ? the Finding object (SUPPORTED BY / CONTRADICTED BY / CONTRACT BASIS / OBLIGATION / EVENT / CAUSATION / QUANTUM / CONFIDENCE / HUMAN REVIEW / STATUS) and the mechanical engine that assembles one from the causation, obligation, timeline, and quantum packages that already existed ? plus two verification functions that re-derive a Finding's claims from source rather than trusting them.

13 commits, 21 new/changed files, ~5,300 lines, 131 new tests, all passing on first or second run (three real bugs were caught by the tests themselves and fixed before commit ? see below).
Race detector, go vet, gofmt, the full repository test suite, and pkg/insurance/guardrails' repo-wide structural scans (no opaque confidence score, no verdict/determination field, anywhere in the insurance domain) all pass clean as of this report.

WHAT WAS REUSED, NOT REBUILT (THE SURVEY'S OWN FINDINGS)

A dedicated research pass read the actual source ? not just type signatures ? of pkg/insurance/{policy,claim,case,evidence,verification,timeline,obligation,coverage,causation,

quantum,dossier,dossierverify,casestate}, pkg/governance/{lifecycle,hitl,calibration, qualification,envelope}, pkg/moat/{causal,contradiction,decision,calibration,reliability, evidencegraph,provenance,entity}, and pkg/explanation. The result, in CRE's own pipeline order:

CRE stage	Reused as-is	Verdict
Contract	pkg/insurance/policy (Version, Clause, History.EffectiveAt)	Pure reuse
Claim	pkg/insurance/claim (Type registry, Status, never APPROVED/DENIED)	Pure reuse
Case aggregate	pkg/insurance/case (15-state pipeline already close to CRE's own stage order)	Pure reuse
Evidence	pkg/insurance/evidence, pkg/evidence/*	Pure reuse
Provenance	pkg/moat/provenance, pkg/evidence/provenance	Pure reuse
Verification	pkg/insurance/verification's four gates (coverage/quantum/preservation/human-review)	Pure reuse
Fact reconstruction	pkg/insurance/timeline (Event, Detector, Condition/Compare)	Pure reuse
Contract-to-fact mapping	pkg/insurance/obligation, pkg/insurance/coverage	Pure reuse
Causation	pkg/insurance/causation (HypothesisSet, Explain, mechanically-derived Status)	Pure reuse
Quantum	pkg/insurance/quantum (Compute, DetectDiscrepancy, Amount as int64 minor units)	Pure reuse
Human review	pkg/governance/hitl (MachineDecision + GovernedOutcome)	Pure reuse
Evidence dossier	pkg/insurance/dossier, pkg/insurance/dossierverify	Pure reuse

For VTECP-001's five capabilities:

Capability	Status before this round	This round
1. Immutable Evidence Manifest	Did not exist	Built ? pkg/evidence/manifest
2. Ontology Layer	A *different*, narrower pkg/kernel/ontology existed (generic meta-schema registry, no fixed vocabulary, no ExecuteAction pipeline)	Built ? pkg/ontology, documented why it is not a duplicate
3. RBAC+ABAC+Purpose Binding Authorization	RBAC+ABAC+ReBAC existed in pkg/authz; Purpose Binding did not	Extended pkg/authz
4. Cryptographically Verifiable Audit Ledger	A hash-chained linear ledger existed (pkg/platform/audit); Merkle inclusion proofs and anchoring did not	Extended pkg/platform/audit
5. AI/Model Lifecycle Governance	Model lifecycle gating existed (pkg/governance/lifecycle); a generic, cross-domain inference audit record did not	Built pkg/inference on top of the existing registry

NEW AND EXTENDED CODE, BY PHASE

PHASE 1?2 ? PKG/CANONICAL/JCS AND PKG/EVIDENCE/MANIFEST

pkg/canonical/jcs implements RFC 8785 (the canonicalization VTECP-001 §7 names explicitly as the correction to a hand-rolled JSON-sorting proposal): UTF-16 code-unit key ordering, correct
string escaping, and ECMAScript Number::toString-compatible number formatting for every number
shape this codebase actually produces (64-bit integers and floats in [-1e15, 1e15]) ? with an
honest, documented refusal (ErrUnsupportedNumber) for magnitudes outside that verified range,
rather than a silent, possibly non-interoperable guess. A real bug was caught by its own test
suite before commit: encodeNumber initially fell straight from int64 to float64, so a

legitimate uint64 tick above math.MaxInt64 was wrongly refused; fixed by trying an exact uint64 parse first.

pkg/evidence/manifest is the Immutable Evidence Manifest: a seven-state lifecycle (DRAFT?INGESTED?INTEGRITY_ASSESSED?PROVENANCE_COMPLETE?READY_FOR_FINALIZATION?FINALIZED, plus SUPERSEDED), a hash-linked custody-event chain, and version history that is never edited in place ? Supersede always creates version N+1, leaving version N provably untouched. A real bug was caught by its own tests: Advance originally computed the manifest's self-referential hash *before* setting FinalizedAt, so every finalized manifest's hash verification failed, including genuinely untampered ones; fixed by reordering the two operations.

35 tests (17 + 18 across the two packages' own test files, verified by direct count).

PHASE 3 ? PKG/ONTOLOGY

The generic object/link/action registry: VTECP-001 §11's 19 core object types plus the framing memo's 14 additional entity types, §13's 9 core link types plus 10 more, tenant-scoped identity distinct from database row identity, and the mandated 5-stage ExecuteAction pipeline (Policy?Validation?Execution?StateTransition?Audit) every ontology mutation runs through. A pre-existing pkg/kernel/ontology (a generic meta-schema/instance validator with no fixed vocabulary) was found during implementation; the two are not duplicates, and the package doc comment records why extending the older package was considered and rejected rather than silently building a second one. 22 tests, all passing on first run ? no bugs surfaced this time.

PHASE 4 ? PKG/AUTHZ EXTENDED

Rule gained Purposes (wildcard-matched, same semantics as Roles/Actions/Resources); Request gained Purpose and Tenant. A rule with no Purposes is purpose-agnostic and evaluates identically to every Document written before the field existed ? additive, not breaking. A new PolicyDecision type is the canonical, hashable, auditable record of one Can() evaluation, with a deterministic DecisionID/PolicyInputsHash derived via jcs.Hash (never a random UUID) so identical inputs against the identical active policy version produce the identical decision ID. Engine.CanRecorded wraps Can() and, when an AuditStore is attached, mirrors the decision into the shared ledger ? including *refused* evaluations, so denials are part of the audit trail, not just grants. 10 new tests, all passing on first run.

PHASE 5 ? PKG/PLATFORM/AUDIT EXTENDED

MerkleRoot/GenerateInclusionProof/VerifyInclusionProof/VerifyRecordInclusion implement RFC 6962 (Certificate Transparency)'s Merkle Tree Hash construction over AuditRecord.Hash values ? domain-separated leaf/node hashing chosen specifically to defeat the standard second-preimage attack a naive pairwise tree is vulnerable to. Anchorer is an interface for committing a root to an external reference; LocalAnchorer is the only implementation shipped, an explicit in-memory simulator whose every receipt self-identifies as such (AnchoredBy: "LocalAnchorer(simulator, not a real external anchor)") ? VERIQO has no real external anchoring integration today, and this says so rather than implying one. AuditStore.Checkpoint/VerifyCheckpoint seal and independently re-verify a Merkle root over a claimed record range. 16 new tests (verified by direct count ? an earlier commit message in this round's own history overstated this as 24; the number here is a fresh grep -c count,

not
a repeated claim), across leaf counts 1 through 33 (both exact-power-of-two and uneven-split recursion branches), tamper detection at every layer, and an explicit test proving VerifyCheckpoint (range/root integrity) and the pre-existing Auditor.VerifyChain (payload-to-hash integrity) are deliberately complementary, not redundant.

PHASE 6 ? PKG/INFERENCE AND PKG/INSURANCE/FINDING

pkg/inference.Recorder is the only writer of InferenceTrace records, and it refuses to accept one for a model that is not the ACTIVE version of its model ID at the claimed tick, per a
bound lifecycle.Registry's own Binding resolution ? this is VTECP-001's core Capability 5 rule ("AI output never becomes evidence authority directly") enforced structurally: an inference
from a DRAFT, unapproved, deprecated, or unknown model is rejected outright, not recorded-with-a-flag. 12 tests.

pkg/insurance/finding.Finding implements CRE's own schema exactly. It is deliberately a citation layer, not a new analysis engine ? every field cites a value some other package already
computed. Status is derived-only (Evaluate always overwrites whatever a caller supplied): a Finding missing even one required field is CANDIDATE, never FINDING. The first draft used an opaque Confidence float64; the repository's own pkg/insurance/guardrails scan (TestNoOpaqueConfidenceScoreAnywhereInTheInsuranceDomain) correctly caught this as a real violation of the domain's established discipline and it was replaced with ConfidenceBasis causation.Status ? the same mechanically-derived classification causation.Hypothesis.Status already computes, reused rather than reinvented. A reflection-based test (TestFindingHasNoVerdictField) enforces that Finding never grows a Verdict/Liable/Guilty/Winner/ApprovedAmount field. 11 tests.

PHASE 7 ? CROSS-SYSTEM INTEGRATION PROOF

One test, TestVTECPCapabilitiesIntegrateAsOneSystem, threads a small real maritime cargo-damage scenario through all five capabilities plus the Finding gate in a single shared *audit.AuditStore, confirming ? by scanning the resulting ledger, not by assertion ? that ontology, authz, and inference all mirror into the same ledger; that a purpose-bound rule denies
the identical request when no purpose is declared; that a model must be walked through its full
DRAFT?ACTIVE lifecycle before an inference trace is accepted; that a real causation.HypothesisSet
(two competing hypotheses, real evidence) drives a Finding that only reaches StatusFinding once
every field is present; and that the InferenceTrace's own audit record's inclusion is proven via
GenerateInclusionProof/VerifyRecordInclusion *without* needing the rest of the ledger, with the proof's root checked against a sealed Merkle checkpoint. Two real fixture bugs (missing ResponsibleParty/Status on a test Obligation) were caught and fixed while writing it. 1 test.

PHASE 8 ? PKG/INSURANCE/CRE: THE CASE RESOLUTION ENGINE CORE

The one genuinely missing mechanical step: turning a completed causation.HypothesisSet into Finding candidates. CandidateHypotheses filters to only SUPPORTED/PARTIALLY_SUPPORTED hypotheses (an empty result ? nothing supported ? is a legitimate, honestly-reported outcome,
never an error). BuildFinding copies SupportedBy/ContradictedBy/ConfidenceBasis verbatim from the real hypothesis, derives Alternatives mechanically as every *other* hypothesis in the

set (never hand-picked), and uses causation.Explain's own hedged narrative ? inventing none of
it. 7 tests.

Two verification functions close the gap BuildFinding's own mechanical path doesn't structurally
prevent ? a caller hand-constructing a Finding directly, bypassing BuildFinding entirely:

- VerifyFindingProvenance confirms a Finding's cited SourceInferenceTraceID, if any, names a real, hash-verified trace ? not just a string that happens to be present. 5 tests.
- VerifyFindingAgainstHypothesis re-derives SupportedBy/ContradictedBy/ConfidenceBasis from the *real* hypothesis in a HypothesisSet (never trusting the Finding's own claim) and confirms they match ? the CRE-level equivalent of pkg/insurance/verification's re-verify-by-recomputing gates, applied to the causation citation. 5 tests, including one that hand-forges a Finding, recomputes its own hash to stay internally consistent (exactly what
a real attacker capable of forging one would do), and confirms it still reaches StatusFinding
on its own terms ? proving that internal hash consistency alone is *not* sufficient proof, which is the entire reason this function exists.

PHASE 9 ? GOLDEN CASES AND ADVERSARIAL TESTS

Two further golden-case scenarios (a crude-oil cargo contamination dispute; a warehouse-fire electrical-fault-vs-arson claim) exercise GenerateFindings ? the engine-level entry point, not
the lower-level BuildFinding the unit tests use ? in domains distinct from Phase 7's maritime
example, confirming the engine generalizes rather than being tuned to one worked case. 2 tests.

Six adversarial tests target the CRE spec's own "6 MUST NOT" list and VTECP-001's AI-governance
rule under actual pressure: a zero-evidence hypothesis cannot be forced into a Finding; a hand-forged Finding is caught by re-derivation against real data even after being made internally
self-consistent; an inference from a never-approved model is refused *and never touches the audit
ledger at all*; Purpose Binding cannot be bypassed by holding the correct role alone; policy denial in the ontology's 5-stage pipeline halts both the mutation and the audit entry; and tampering a single record contributed by one subsystem breaks chain verification for a ledger
shared across subsystems. 6 tests.

Golden-case domain data (case/evidence IDs, amounts, narratives) is clearly synthetic test fixture data, not a claim about any real case ? consistent with this repository's discipline throughout every prior round.

HONEST SCOPE BOUNDARIES

Consistent with VTECP-001 §58's explicit anti-fabrication rule, the following are stated plainly
rather than glossed over:

- LocalAnchorer is a simulator. VERIQO has no real external anchoring integration (no blockchain, no third-party notarization service, no regulator timestamping authority) today. Every receipt it issues says so in its own AnchoredBy field. A real Anchorer plugs into AuditStore.Checkpoint without any caller-visible change when one exists.
- pkg/inference does not run any AI model. It is a governed audit-trail primitive: it

records that a call happened, to a specific governed model version, with a specific input commitment and output, gated on that model being ACTIVE. It has no opinion on how the output was produced, and makes no claim of running or hosting inference itself.
- jcs.Canonicalize has a bounded, documented number range. Magnitudes outside [-1e15, 1e15] or requiring exponential notation are refused (ErrUnsupportedNumber), not silently mis-encoded, because this implementation has not been verified against the ECMAScript spec's exact behavior at the extreme ends of the IEEE-754 range.
- A ConfidenceBasis can still be hand-fabricated by a caller who bypasses BuildFinding entirely ? this is exactly why VerifyFindingAgainstHypothesis exists as a *separate*, mandatory-to-call re-verification step, not folded silently into Finding.Evaluate itself (which has no access to the real HypothesisSet to check against). A Finding's own StatusFinding alone is not proof of anything; VerifyFindingAgainstHypothesis and VerifyFindingProvenance are.
- The two additional golden cases (crude-oil contamination, warehouse fire) are illustrative scenarios built to exercise the engine across domains, not literal transcriptions of specific
worked examples from the source specification documents ? this round did not have the original CRE docx's own named worked examples (§24/§26/§27, referenced in earlier planning) in
working context by the time golden-case tests were written, and this report says so rather than
presenting reconstructed guesses as verbatim spec content.
- No real counterparty, expert, or regulator was contacted. Every actor name (analyst-1, surveyor-lead-1, ml-engineer-1) in every test is a synthetic fixture label.

VERIFICATION PERFORMED

- gofmt -l across every file changed or added this round: clean.
- go build ./...: clean, at every phase and at the end.
- go vet ./...: clean, at every phase and at the end.
- go test -race on every new/extended package: clean.
- Full repository test suite (go test ./..., all ~150+ packages including test/e2e, test/integration, test/soak, test/stress, test/chaos, test/acceptance): clean, run twice during this round (after Phase 5 and after Phase 6) with no regressions, and once more as
this report's own closing verification.
- pkg/insurance/guardrails' seven repo-wide structural scans (no determination/verdict field,
no opaque confidence score, no forbidden canonical duplicate, no hard-coded vendor judgment, full package coverage): clean.
- 131 new tests this round (verified by direct grep -c '^func Test' count per file, not carried forward from earlier, less-precisely-counted commit messages), all passing.

FILES CHANGED THIS ROUND

13 commits, 0540117..HEAD on claude/l99-gap-coverage-nv70zy:

pkg/canonical/jcs/{jcs.go, jcs_test.go} [new package]
pkg/evidence/manifest/{manifest.go, manifest_test.go} [new package]
pkg/ontology/{ontology.go, ontology_test.go} [new package]
pkg/authz/{authz.go [extended], purpose_decision_test.go} [extended]
pkg/platform/audit/{merkle.go, merkle_test.go} [extended]
pkg/inference/{inference.go, inference_test.go} [new package]
pkg/insurance/finding/{finding.go, finding_test.go} [new package]
pkg/insurance/cre/{engine.go, engine_test.go,
provenance.go, provenance_test.go} [new package]
test/integration/vtecp_cre_integration_test.go [new]
test/integration/cre_golden_cases_test.go [new]

test/integration/cre_vtecp_adversarial_test.go [new]
docs/VERIQO_VTECP_CRE_CLOSURE_REPORT.{md, pdf} [this report]

CONCLUSION

VTECP-001’s five capabilities and CRE’s eleven-stage pipeline are now fully wired, tested, and integration-proven ? five capabilities either newly built or genuinely extended, eleven pipeline stages reused without duplication, one net-new mechanical engine connecting causation output to the Finding gate, and two re-verification functions ensuring a Finding’s claims can be checked against real source data rather than trusted at face value. Nothing in this deliverable asserts a verdict, a liability determination, or automatic court-admissibility; every gate that exists (Finding’s field-completeness check, the model-ACTIVE requirement, Purpose Binding, the audit ledger’s hash chain and Merkle proofs) fails closed, and every honest limitation is stated in this report rather than implied away.